Deep Learning
-- Project proposal --

CNN-Based Semiconductor Wafer Defect Detection Using the WM-811K Dataset

Team 4

Anindita Paul, Brandon Pyle, Anand Rajan, and Brett Rettura

AI 570 − Deep Learning

Spring Semester, 2026

TABLE OF CONTENTS

I. INTRODUCTION *3*
II. Problem Statement *3*
III. Challenges *4*
IV. Related Works *4*
V. Data Description *4*
VI. Deep Learning Methodology *5*
VII. Expected Results *6*
VIII. Impact and Significance *7*
IX. Limitations and Future Work *7*
X. Reproducibility *7*
XI. References to Data Sources *7*
XII. Justification for Using Existing Code *7*

CNN-Based Semiconductor Wafer Defect Detection Using the WM-811K Dataset

Anindita Paul, Brandon Pyle, Anand Rajan, and Brett Rettura

I. INTRODUCTION

The cornerstone of the current economy is the semiconductor industry. The projected 2026 annual sales value of this industry is US$975 billion[1], driven by intense demand for AI infrastructure. As device size has continued to shrink in accordance with Moore's Law, with current nodes at 3nm and below, the complexity of the manufacturing processes has increased exponentially. A single semiconductor wafer can contain thousands of individual chips, each worth hundreds or thousands of dollars.

Semiconductor manufacturing combines repetitive, complex, additive, and subtractive processes to assemble integrated circuit (IC) chips on a silicon wafer, making it among the most complex manufacturing processes [2]. It can be classified as a) the front-end process and b) the backend process. In the front-end process, multiple circuit layers are deposited via the following key steps. A) Oxidation and deposition: The layers of insulating material, such as SiO2, are deposited via oxidation. Using Chemical vapor deposition (CVD) or physical vapor deposition (PVD), metals are deposited on the wafer surface to form a conductive layer. B) Precise patterns are defined by means of photolithography and etching. C) Finally, Ion implantation or diffusion introduces impurities to alter the electrical properties, such as conductivity, to form transistors or other devices.

The effectiveness of the front-end process is assessed through wafer testing to verify that each wafer operates properly. This is considered a backend process. In this wafer test process, the number of functional chips that can be used from a wafer is counted, and the ratio of functional chips to total chips is defined as the yield. Yield is the key indicator of semiconductor manufacturing process quality. Due to technological scaling, the minimum feature size is now below 3 nanometers. The demand for high-speed, multifunctional electronic devices drives the need to integrate billions of transistors per chip into a highly dense arrangement. As a result, it is difficult to identify a single microscopic defect that is invisible to the naked eye, which can cause an entire die to fail. These defects manifest as distinct spatial patterns on the wafer and can arise from hundreds of complex process steps, such as lithography, etching, deposition, and polishing. These systematic defects [3, 4] that occur in the wafer can be categorized into the following classes: i) Center, which is concentrated at the center of the wafer, and is caused by the failure of the thin-film deposition step [5]. ii) Donut generally refers to a ring-shaped pattern created due to the redeposition of dissolved photoresist back to the wafer surface or issues with chemical dispense. iii) Edge-ring forms along the edge due to abnormal etching operations. iv) Loc (Local) is a small, localized cluster defect created due to non-uniform developer application, drastic pH changes during rinsing, or low dissolution rates in specific areas. v) Edge-Loc is a localized defect that appears along the edge of the wafer. These defects typically happen from failures in edge-sensitive processes, particularly during the middle-of-line (MoL) or backend, where handling and coatings are complex. vi)Near-full, a high density of defects covering most of the wafer from a critical process failure, vii) Scratches are linear or scratch-shaped patterns of defects, caused by physical damage or debris. Finally, viii) a wafer with no detected defect. In addition to this systematic error, there can be random errors, irregularly distributed defects scattered across the wafer without a specific pattern, typically caused by contaminants in the cleanroom. Identification and classification of defect patterns in wafer maps can provide engineers with useful information to detect the root causes of failures in wafer manufacturing, thereby improving wafer production yield[6].

II. PROBLEM STATEMENT

The semiconductor manufacturing industry faces significant challenges in accurately and efficiently detecting and classifying wafer defects during production. Current manual inspection methods are time-consuming, subjective, and error-prone, leading to inconsistent defect classification, delayed production cycles, and substantial yield losses. This project aims to develop an automated detection and classification system using multiple Convolutional Neural Network (CNN)[7] architectures, such as Custom CNN, ResNet and EfficientNet[8] on the WM-811K dataset, the largest publicly available dataset comprising 811K wafer maps across 9 classes. Our deep learning model mitigates class imbalance and noise to achieve accurate pattern recognition, treating wafer maps as images. The successful implementation of this automated detection and classification system will enhance manufacturing yield in semiconductor production facilities. The system will eliminate human variability and provide uniform defect classification 24/7. It will enable early fault detection to prevent defect propagation, reduce labor costs, and increase yield, thereby ultimately increasing industry revenue.

III. Challenges

Several challenges arise in applying deep learning to wafer-map defect detection:

1. **Severe class imbalance.** The WM-811K dataset[9] is dominated by the "none" class (~85% of labeled samples), while the rarest defect class (Near-full) contains only 149 examples. Standard classifiers trained without mitigation strategies will overwhelmingly predict the majority class.

2. **Computational constraints.** Training deep CNNs on a dataset of 172,950 wafer maps is computationally expensive. In this study, we are limited to CPU-only hardware, necessitating architectural compromises such as pre-resizing to 96×96 and capping the number of training samples per epoch.

3. **Domain gap for transfer learning.** Pretrained ImageNet models have learned features from natural photographs, whereas wafer maps are synthetic, low-resolution, binary-like images. The transferability of such features to the wafer-map domain is not guaranteed.

4. **Defect pattern diversity.** The nine defect classes exhibit highly variable spatial morphologies. Some patterns (e.g., Edge-Ring) are visually distinctive, while others (e.g., Loc vs. Random) can be difficult to distinguish even for experienced engineers.

5. **Legacy data format.** The WM-811K dataset is stored as a Python 2 pickle file, which requires encoding workarounds and module-path remapping in modern Python 3 environments.

IV. Related Works

Wu et al.[9] introduced the WM-811K dataset and proposed a similarity-based ranking approach using density-based spatial features for root-cause analysis. Nakazawa and Kulkarni[10] subsequently applied CNN architecture to WM-811K, coupling defect classification with image retrieval. Yu et al.[11] proposed a hybrid framework combining a densely connected CNN with a deep forest, leveraging transfer learning for minority classes. Saqlain et al.[12] introduced a voting ensemble classifier that aggregated predictions from multiple base learners. Kahng and Kim[13] explored self-supervised representation learning for wafer bin map classification, demonstrating that pre-text tasks tailored to the spatial structure of wafer maps can yield informative features without labeled data. Kim et al.[14] proposed a single-shot detector for identifying mixed-type defect patterns. Chen et al.[15] developed an attention-based method for wafer-map defect pattern detection.

Foundational deep-learning work underpinning our approach includes ResNet with skip connections for training very deep networks, EfficientNet with compound scaling, batch normalization, dropout, and Grad-CAM for visual explanations. Transfer learning from ImageNet has proven effective across industrial imaging tasks.

Despite substantial progress, several gaps remain, including that many prior studies report only overall accuracy without per-class metrics for imbalanced data. Few provide visual explanations of model decisions, and comparisons across modern efficient architectures are underrepresented. Our work addresses these gaps by systematically comparing three CNN architectures under identical conditions, employing multiple class-imbalance strategies, and providing Grad-CAM visualizations.

V. Data Description

A. The WM-811KDataset

The WM-811K dataset[9] is distributed as a serialized Python pickle file (LSWMD.pkl) and contains 811,457 wafer map records collected from real semiconductor production lines. Each record includes a two-dimensional array, where each element takes one of three values: 0 (background), 1 (normal die), or 2 (defective die), along with metadata such as the lot name, wafer index, and failure-type label.

Only the subset of wafers with an assigned failure-type label is used for supervised classification. After filtering, approximately 172,950 labeled wafer maps remain, categorized into the following nine classes of systematic defect patterns:

- **Center**: concentrated at the wafer center, caused by failure in thin-film deposition.
- **Donut**: ring-shaped pattern from photoresist redeposition or chemical dispense issues.
- **Edge-Ring**: along the wafer edge from abnormal etching.
- **Loc (Local)**: a small cluster from a non-uniform developer application or pH changes.
- **Edge-Loc**: localized defect along the wafer edge from edge-sensitive processes.
- **Near-full**: high-density defects covering most of the wafer from a critical process failure.
- **Scratch**: linear patterns from physical damage or debris.
- **Random**: irregularly distributed defects from cleanroom contaminants.
- **None**: no systematic spatial pattern detected.

B. Class Distribution

Table 1 shows a severe class imbalance: the "none" class accounts for over 85% of labeled samples, whereas Near-full contains only 149 examples (~990:1).

TABLE I
CLASS DISTRIBUTION OF LABELED WAFER MAPS

Defect Class	Count	%
None	147,431	85.24
Edge-Ring	9,680	5.60
Edge-Loc	5,189	3.00
Center	4,294	2.48
Loc	3,593	2.08
Scratch	1,193	0.69
Random	866	0.50
Donut	555	0.32
Near-full	149	0.09
Total	**172,950**	**100.00**

C. Preprocessing

Each wafer map is preprocessed as follows: (1) resized to 96×96 pixels using nearest-neighbor interpolation to preserve the discrete die values; (2) pixel values normalized to [0,1] by dividing by 2; and (3) the wafer map array is normalized and replicated across three channels to produce a $3 \times 96 \times 96$ tensor compatible with pretrained ImageNet models.

VI. DEEP LEARNING METHODOLOGY

A. Problem Classification

The problem is formulated as a supervised multi-class classification across nine defect pattern classes. The labeled dataset is partitioned into training (70%), validation (15%), and test (15%) subsets via stratified sampling to maintain class proportions. To address the severe class imbalance (none: 85.24%, near-full: 0.09%), a WeightedRandomSampler[16] assigns each training sample a weight inversely proportional to its class frequency as given in (1).

$$w_i^{sample} = \frac{1}{n_{(c)i}} \tag{1}$$

Here, $n_c(i)$ denotes the training count for class $c(i)$ of sample i. This ensures minority classes receive proportionally greater sampling probability during training.

B. CNN Architecture

The proposed custom CNN architecture comprises five convolutional blocks designed to extract hierarchical features from wafer map spatial patterns. Each block follows the structure Conv2d (3x3 kernel, stride 1, padding 1) → BatchNorm2d → ReLU → MaxPool2d (2x2), with channel progression 3 → 32 → 64 → 128 → 256 → 512. This progressive expansion enables the network to capture increasingly complex spatial features at each layer.

The classification head employs AdaptiveAvgPool2d to reduce the spatial dimensions to 1x1, followed by a flatten operation. Two fully connected layers map the extracted features to the output space: the first layer (512 → 256) incorporates ReLU activation and dropout ($p = 0.5$) for regularization, while the final layer (256 → 9) applies softmax activation to produce class probabilities. The architecture contains approximately 2.55M trainable parameters, providing sufficient capacity for nine-class classification while maintaining computational efficiency compared to standard architectures (VGG16 [17]: 138M, ResNet-18 [7]: 11M parameters).

C. Data Augmentation

To mitigate class imbalance and improve generalization, several augmentation techniques are applied during training. Random horizontal and vertical flipping ($p = 0.5$) introduces rotational invariance, while random rotation ($\pm 15°$) and random affine translation (up to 5% of image dimension) account for natural variations in defect positioning. Additionally, Gaussian noise injection ($\sigma = 0.01$, $p = 0.3$) improves robustness to sensor noise and minor image artifacts. These transformations preserve the spatial characteristics of defect patterns while expanding the effective training set size, particularly benefiting minority classes.

D. Training Configuration

The model is trained with the Adam optimizer [18] and a class-weighted loss. Training hyperparameters are specified in Table 2.

TABLE 2
TRAINING HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
LR (Custom CNN)	1×10^{-3}
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=5)
Batch Size	64
Number of Epochs	100*
Weight Decay (L2)	1×10^{-4}
Dropout Rate	0.5
Momentum (β_1, β_2)	0.9, 0.999
Random Seed	42

*CPU training requires ~100 epochs to achieve convergence comparable to 25 GPU epochs due to slower per-epoch processing reduced effective batch utilization. GPU-accelerated training accounts for a ~4x speedup in backpropagation and batch processing.

E. Loss Function

Class-weighted categorical cross-entropy penalizes the misclassification of minority classes, as given in (2).

$$L = -\sum_{c=1}^{9} w_c \cdot y_c \cdot \log\left(p_c\right) \tag{2}$$

Here, w_c is the weight for class c. y_c is the ground truth one-hot encoded label, and $\widehat{p_c}$ is the predicted probability. Class weights are computed as given in (3).

$$w_c = \frac{N}{C \cdot n_c} \tag{3}$$

where N is the total number of training samples, $C = 9$ is the number of classes, and n_c is the number of samples in class c .

The ReduceLROnPlateau scheduler monitors validation loss and reduces the learning rate by 0.5 when no improvement is observed for 5 consecutive epochs. Early stopping with a patience of 15 epochs prevents overfitting, where CPU hardware limitations are not a constraint.

F. Evaluation Metrics

Given the severe class imbalance, multiple complementary metrics are employed. Overall accuracy provides a baseline measure of correctly classified wafer maps, though it can be misleading in imbalanced datasets. The macro-averaged F1-score, computed as the arithmetic mean of per-class F1-scores, provides equal weight to all classes regardless of frequency and serves as the primary performance indicator. The weighted F1-score accounts for class support to reflect real-world distribution. Per-class precision, recall, and F1-score are computed independently for each of the nine defect classes to identify class-specific performance patterns. Confusion matrices visualize classification patterns and inter-class confusion, while ROC-AUC employs a one-vs-rest multi-class extension to assess discriminative capability across all classes.

G. Model Interpretability via Grad-CAM

Gradient-weighted Class Activation Mapping generates visual explanations by computing gradients of the target-class score with respect to the activations of the final convolutional layer. The Grad-CAM heatmap for the class c is given in (4).

$$L^c_{Grad-CAM} = ReLU\left(\sum_k^{\alpha_k^c} A^k\right) \tag{4}$$

Here, α_k^c represents the importance weight of feature map k for class c , obtained via global average polling of $\frac{\partial y^c}{\partial A^k}$, and A^k denotes the k-th feature map activation. The ReLU activation function ensures that only positive contributions are visualized. These class-discriminative localization maps validate that the network attends to actual defect regions rather than spurious correlations.

H. Implementation

The methodology is implemented in PyTorch [19] 1.9.1 with NumPy 1.21, OpenCV 4.5.3, and scikit-learn [20] 1.0.2. Training is designed for NVIDIA GPU with CUDA 11.1 support. An alternative MATLAB R2021b implementation using the Deep Learning Toolbox is provided for compatibility. Using a fixed random seed (seed=42) ensures reproducibility.

VII. Expected Results

A. Anticipated Performance Metrics

Based on similar studies employing CNN architectures on WM-811K and preliminary experiments, classification accuracy of 95-97%, macro-averaged F1-score of 0.90-0.93, and weighted F1-score of 0.95-0.97 are anticipated benchmarks.

Categorizing this by class performance, Table 3 presents the expected accuracy and notes for each class.

Primary misclassifications are anticipated between visually similar patterns. Edge-Local and Edge-Ring both exhibit edge-localized features, making differentiation challenging. Similarly, the center and Donut patterns both exhibit a central concentration that can be confusing. The Loc and Random classes exhibit scattered distributions across the wafer, potentially leading to misclassification between these categories.

The dynamics of the training of this model are such that convergence is expected within 50-70 epochs with validation loss plateauing. The ReduceLROnPlateau scheduler should trigger 2-3 learning rate reductions. Early stopping is anticipated around epoch 85-90 if the full 100-epoch schedule is executed.

TABLE 3
Per-Class Performance Expectations

Class Category	Expected Accuracy	Notes
High- frequency (None, Random)	>98%	Abundant training samples
Medium-frequency (Center, Edge-Ring, Loc)	93-96%	Moderate representation
Low-frequency (Donut, Near-full)	88-92%	Limited training examples

B. Success Criteria

I. Quantitative Thresholds

- Overall classification accuracy of $\geq 95\%$
- Macro-averaged f1-score of ≥ 0.90 (ensuring minority class performance)
- Weighted F1-score of ≥ 0.96
- Inference time < 50 ms per wafer map on GPU

II. Qualitative Validation

Grad-CAM heatmaps must demonstrate focus on actual defect regions to validate that the model learns genuine spatial patterns rather than artifacts. The model must maintain accuracy exceeding 90% under geometric perturbations, including rotation ($\pm 20°$) and translation ($\pm$ 15%) to ensure robustness beyond the training distribution. Cross-validation stability is assessed by requiring a standard deviation below 2% across 5-fold cross-validation, confirming consistent performance independent of data partitioning.

C. Comparative Baseline Analysis

To contextualize performance, evaluation will be conducted against the baseline methods, whose hypothetical comparisons are listed in Table 4.

The proposed architecture aims to achieve competitive accuracy with significantly improved computational efficiency.

VIII. Impact and Significance

Automated classification enables rapid root-cause analysis, potentially reducing defect rates by enabling earlier detection of systematic manufacturing issues. Elimination of variability in manual inspection provides consistent 24/7 defect classification. Statistical analysis of defect patterns facilitates predictive maintenance and process optimization, directly improving manufacturing yield.

TABLE 4

Anticipated Comparative Performance

Method	**Expected Accuracy**	**Inference Time**	**Notes**
SVM with Handcrafted Features	85-88%	~30ms	Manual feature engineering baseline
Shallow CNN (2 blocks)	90-92%	~25ms	Validates depth necessity
Fine-tuned VGG16 [17]	93-95%	500-750ms	High computational cost
Proposed 5-Layer CNN	**95-97%**	**<50ms**	**Optimal accuracy-efficiency trade-off**

The lightweight architecture (~2.5M parameters) enables edge deployment of inspection equipment. Grad-CAM integration provides interpretability crucial for adoption in risk-averse manufacturing environments. The weighted loss and augmentation framework provides a reproducible approach to addressing severe class imbalance across industrial inspection domains.

Effective use of the publicly available WM-811K dataset supports reproducible research. The comprehensive evaluation framework establishes best practices for wafer defect classification. Reported results serve as benchmarks for future comparative studies.

IX. Limitations and Future Work

Training is limited to single-facility WM-811K data, and cross-facility generalization requires domain adaptation techniques to account for variations in manufacturing processes and equipment. Rare defect patterns with occurrence below 0.5% may be inadequately represented in the training set, limiting detection capability for novel failure modes. The model does not incorporate temporal information such as defect evolution across wafer lots, which could provide additional predictive signals. Finally, the model is trained on 96x96 resized images. Higher resolution may capture finer defect details but at increased computational cost.

Multi-task learning frameworks could simultaneously predict defect class and severity. Ensemble methods that combine models with different initializations may improve performance in minority classes. Active learning with uncertainty-based sampling could optimize labeling effort for rare defects. Model quantization (INT8) and pruning could enable real-time edge deployment.

X. Reproducibility

All experiments will be repeated with several random seeds, and the results will be reported as the mean and standard deviation. McNemar's test will assess statistical significance ($p < 0.05$) of performance differences. Upon completion, the trained model weights and inference code will be publicly released, along with comprehensive documentation of the rationale for hyperparameter selection.

XI. References to Data Sources

A. Patel, "WM-811K WaferMap Dataset," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/code/ashishpatel26/wm-811k-wafermap

XII. Justification for Using Existing Code

The code used in this project was developed using a combination of our own code, online resources, course

materials, and AI coding tools. This approach ensured that our code adhered to industry standards and course expectations. The result is a robust Jupyter Notebook file that enables us to apply the models described in this proposal to the dataset, visualize the data, and review the results.

REFERENCES

[1] S. Engineering, "Chip Industry Week in Review," ed, 2025.

[2] H. Kahng and S. Kim, "Self-Supervised Representation Learning for Wafer Bin Map Defect Pattern Classification," *IEEE Transactions on Semiconductor Manufacturing,* vol. PP, pp. 1-1, 11/16 2020, doi: 10.1109/TSM.2020.3038165.

[3] S. Chen, M. Liu, X. Hou, Z. Zhu, Z. Huang, and T. Wang, "Wafer map defect pattern detection method based on improved attention mechanism," *Expert Systems with Applications,* vol. 230, p. 120544, 2023/11/15/ 2023, doi: https://doi.org/10.1016/j.eswa.2023.120544.

[4] L. Xuen *et al.*, "The Classification of Wafer Defects: A Support Vector Machine with Different DenseNet Transfer Learning Models Evaluation," 2023, pp. 304-309.

[5] Y. Chen, T. Yuan, S. J. Bae, and Y. Kuo, "Two-level differential burn-in policy for spatially heterogeneous defect units in semiconductor manufacturing," *Computers & Industrial Engineering,* vol. 162, p. 107768, 2021/12/01/ 2021, doi: https://doi.org/10.1016/j.cie.2021.107768.

[6] S.-K. S. Fan, P.-C. Chen, C.-H. Jen, and K. Sethanan, "Auto-Labeling for Pattern Recognition of Wafer Defect Maps in Semiconductor Manufacturing," *Journal of Manufacturing Science and Engineering,* vol. 146, no. 7, 2024, doi: 10.1115/1.4065118.

[7] V. Jadeja, A. L. N. Rao, A. Srivastava, S. Singh, P. Chaturvedi, and G. Bhardwaj, "Convolutional Neural Networks: A Comprehensive Review of Architectures and Application," in *2023 6th International Conference on Contemporary Computing and Informatics (IC3I)*, 14-16 Sept. 2023 2023, vol. 6, pp. 460-467, doi: 10.1109/IC3I59117.2023.10397695.

[8] A. N. Handayani, T. Andriyanto, D. F. Azizah, M. Z. Wiryawan, and H. A. Rosyid, "Comparison of ResNet-50 and EfficientNet-B0 Method for Classification of Indonesian Sign Language System (SIBI) Using Multi Background Dataset," in *2024 International Conference on Computer Engineering, Network, and Intelligent Multimedia (CENIM)*, 19-20 Nov. 2024 2024, pp. 1-6, doi: 10.1109/CENIM64038.2024.10882836.

[9] M. J. Wu, J. S. R. Jang, and J. L. Chen, "Wafer Map Failure Pattern Recognition and Similarity Ranking for Large-Scale Data Sets," *IEEE Transactions on Semiconductor Manufacturing,* vol. 28, no. 1, pp. 1-12, 2015, doi: 10.1109/TSM.2014.2364237.

[10] T. Nakazawa and D. V. Kulkarni, "Wafer Map Defect Pattern Classification and Image Retrieval Using Convolutional Neural Network," *IEEE Transactions on Semiconductor Manufacturing,* vol. 31, no. 2, pp. 309-314, 2018, doi: 10.1109/TSM.2018.2795466.

[11] Z. Shen and J. Yu, "Wafer Map Defect Recognition Based on Deep Transfer Learning," *2019 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM),* pp. 1568-1572, 2019.

[12] M. Saqlain, B. Jargalsaikhan, and J. Y. Lee, "A Voting Ensemble Classifier for Wafer Map Defect Patterns Identification in Semiconductor Manufacturing," *IEEE Transactions on Semiconductor Manufacturing,* vol. 32, pp. 171-182, 2019.

[13] H. Kahng and S. B. Kim, "Self-Supervised Representation Learning for Wafer Bin Map Defect Pattern Classification," *IEEE Transactions on Semiconductor Manufacturing,* vol. 34, no. 1, pp. 74-86, 2021, doi: 10.1109/TSM.2020.3038165.

[14] T. S. Kim, J. W. Lee, W. K. Lee, and S. Y. Sohn, "Novel method for detection of mixed-type defect patterns in wafer maps based on a single shot detector algorithm," *Journal of Intelligent Manufacturing,* vol. 33, no. 6, pp. 1715-1724, 2022/08/01 2022, doi: 10.1007/s10845-021-01755-6.

[15] Z. Long, J. Yan, and M. Piao, "Multi-scale guidance diffusion network for wafer map defect recognition," *Expert Systems with Applications,* vol. 267, p. 126134, 2025/04/01/ 2025, doi: https://doi.org/10.1016/j.eswa.2024.126134.

[16] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Transactions on Knowledge and Data Engineering,* vol. 21, no. 9, pp. 1263-1284, 2009, doi: 10.1109/TKDE.2008.239.

[17] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *CoRR,* vol. abs/1409.1556, 2014.

[18] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *CoRR,* vol. abs/1412.6980, 2014.

[19] A. Paszke *et al.*, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019.

[20] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research,* vol. 12, 01/02 2012.